Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

- Convolution ✓
- Pooling
- Fully Connected Layers

2.3 Full CNN Pipeline

2.4 Training CNNs

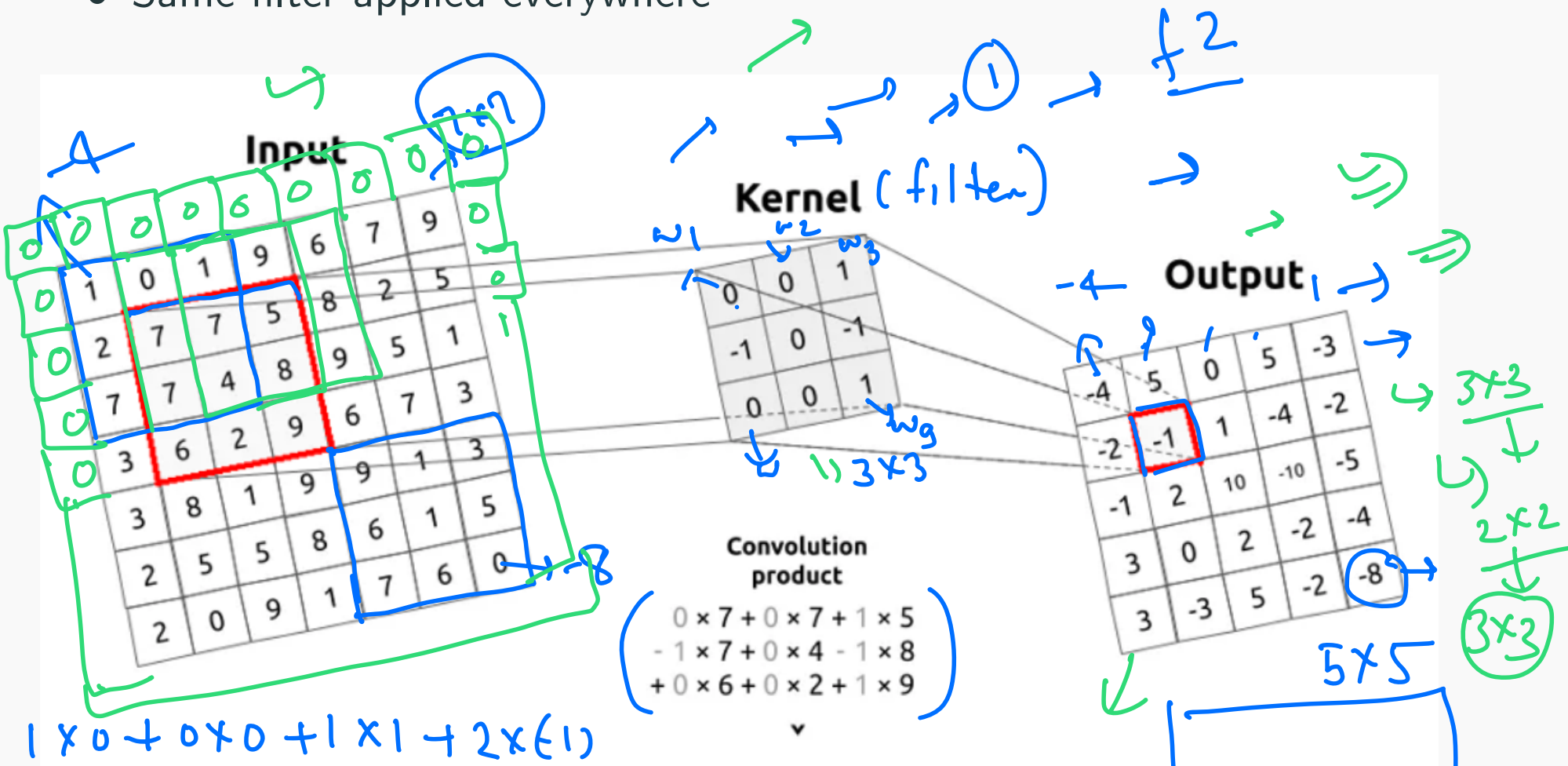
2.5 Practical Tips

2.6 Popular CNN Architectures

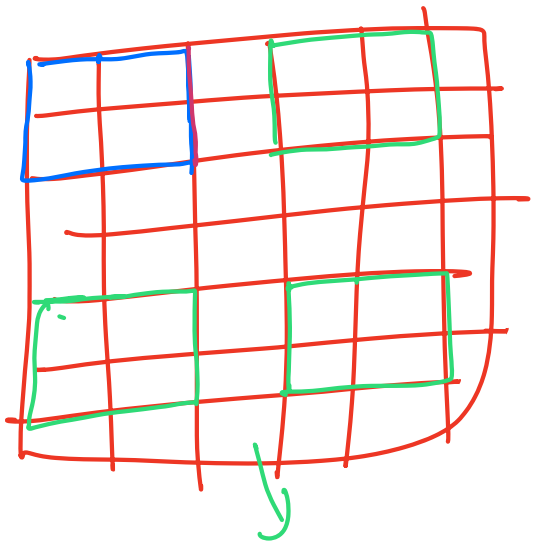
2.7 CNNs Lab

What is Convolution? (Intuition)

- A small filter scans across the image
- Detects patterns like edges, textures
- Same filter applied everywhere

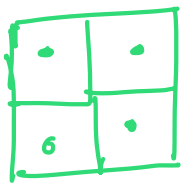


$1\times0+0\times0+1\times1+2\times(-1)$
$+7\times0+(-1)\times7+\ldots=-4$



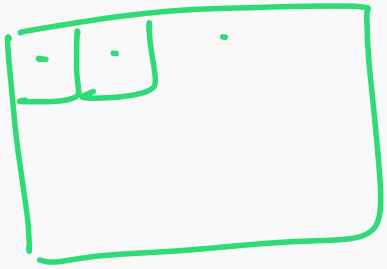
S=3

2×2



Convolution Operation

$$Y[i,j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W[u,v]\, X[i+u, j+v]$$



- W: filter (kernel), learned during training
- X: input image (or previous feature map)
- Y: output feature map (one map per filter)
- Intuition: each output pixel is a *weighted sum* of a local patch

Learn more from: https://developer.nvidia.com/discover/convolution

Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.

Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.
- In general, data can be represented as:

$$X \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_k \times C}$$

where C denotes the number of channels, and each channel is:

$$X[:, :, \ldots, :, c]$$

GrayScal $\Rightarrow X \in R^{H \times W \times 1}$

RGB $\Rightarrow X \in R^{H \times W \times 3}$

$X \in R^{H \times W \times C}$

$X \in R^{D \times H \times W \times 3}$

Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.
- In general, data can be represented as:

$$X \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_k \times C}$$

where C denotes the number of channels, and each channel is:

$$X[:, :, \ldots, :, c]$$

kernel

- **Interpretation:** A channel captures one type of information or feature across the spatial/temporal dimensions.

Channels (Depth)

- A **channel** is one *feature map* along the depth dimension of the data tensor.
- In general, data can be represented as:

$$X \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_k \times C}$$

where C denotes the number of channels, and each channel is:

$$X[:, :, \ldots, :, c]$$

- **Interpretation:** A channel captures one type of information or feature across the spatial/temporal dimensions.
- **Special cases:**
  - **1D data (signals, time series):** $X \in \mathbb{R}^{L \times C}$
    Channel = 1D signal
  - **2D data (images):** $X \in \mathbb{R}^{H \times W \times C}$
    Channel = 2D map (e.g., RGB planes)
  - **3D data (videos, volumetric):** $X \in \mathbb{R}^{D \times H \times W \times C}$
    Channel = 3D volume

Filters (Kernels): How Channels are Produced i

- A **filter/kernel** detects local patterns by sliding over the input
- **Key idea:**
  - Each filter spans **all input channels**
  - **One filter** $\Rightarrow$ **one output channel**
  - K **filters** $\Rightarrow$ K **output channels**
- **1D Convolution (Time Series)**
  - Input: $X \in \mathbb{R}^{L \times C}$
  - One filter: $W^{(m)} \in \mathbb{R}^{k \times C}$
  - Output (one channel):

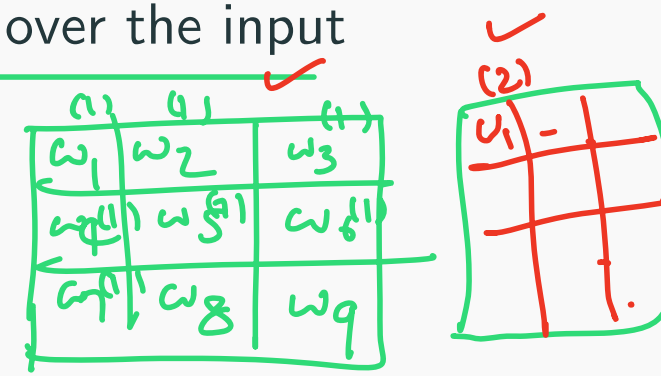
m=1

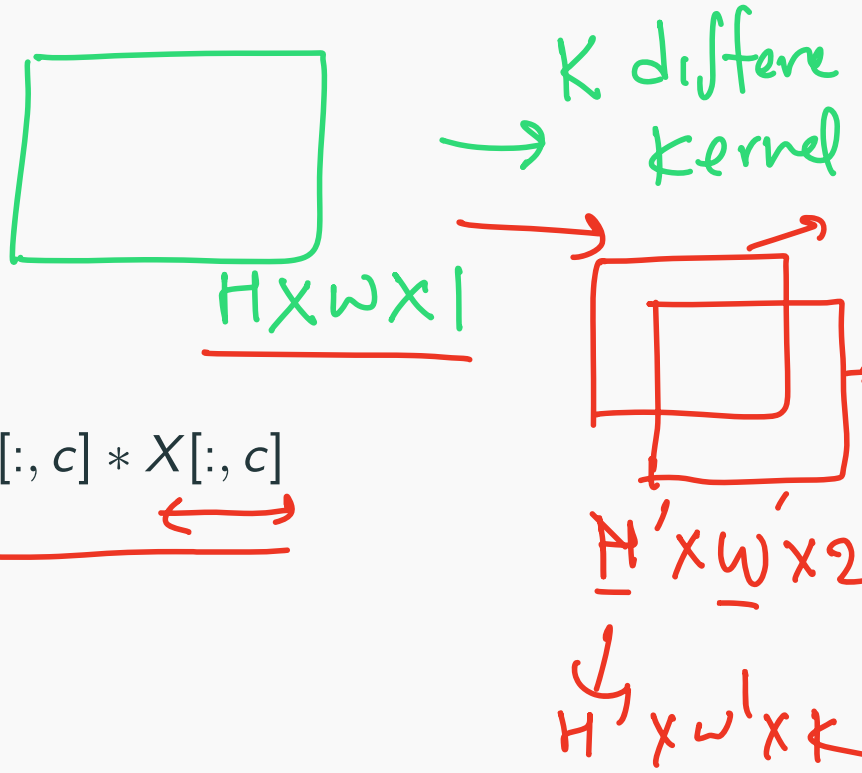
$$Y[:, m] = \sum_{c=1}^{C} W^{(m)}[:, c] * X[:, c]$$

m=1..K

  - Using K filters $\Rightarrow$ $Y \in \mathbb{R}^{L' \times K}$.

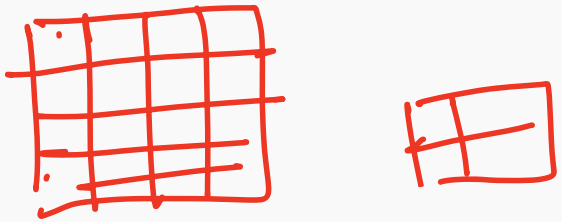
$L' < L$





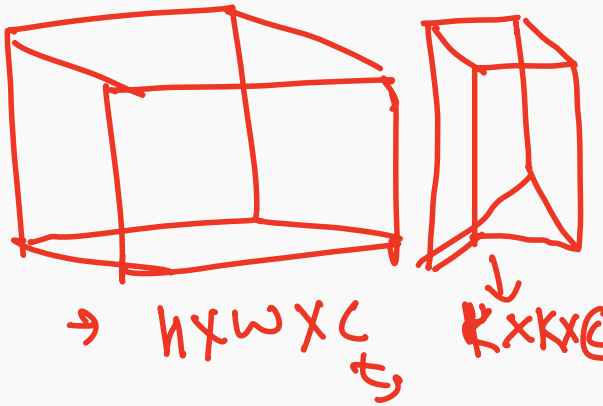
Filters (Kernels): How Channels are Produced ii

- **2D Convolution (Images)** RG
  - Input: $X \in \mathbb{R}^{H \times W \times C}$
  - One filter: $W^{(m)} \in \mathbb{R}^{k \times k \times C}$
  - Output (one channel):



$$\boxed{Y[:,:,m] = \sum_{c=1}^{C} W^{(m)}[:,:,c] * X[:,:,c]}$$

  - Using K filters $\Rightarrow Y \in \mathbb{R}^{H' \times W' \times K}$



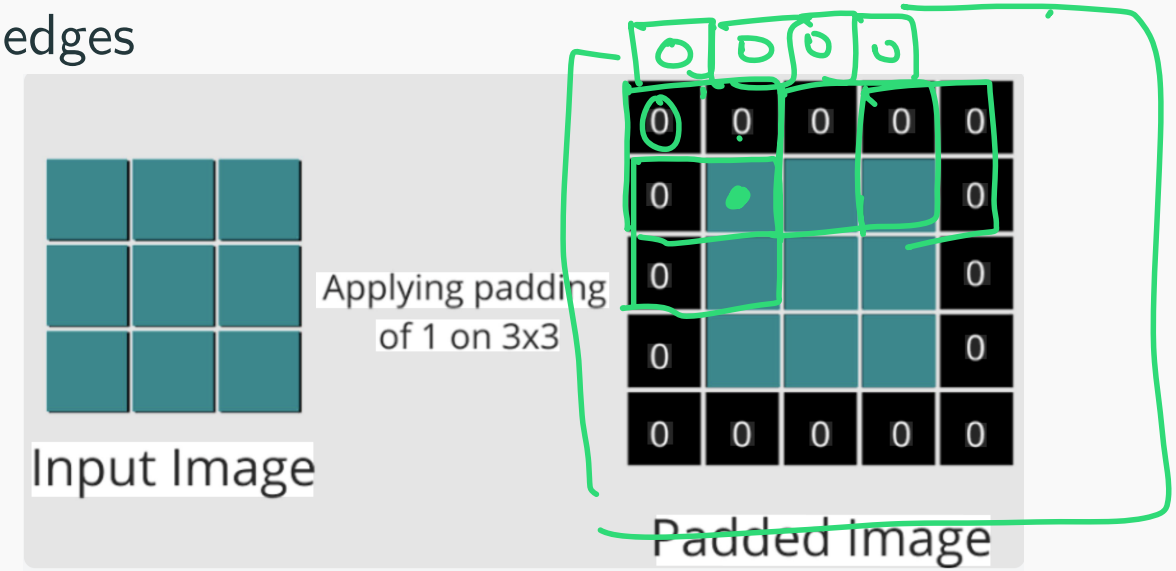
- **3D Convolution (Videos / Volumes)**
  - Input: $X \in \mathbb{R}^{D \times H \times W \times C}$
  - One filter: $W^{(m)} \in \mathbb{R}^{k \times k \times k \times C}$
  - Output (one channel):

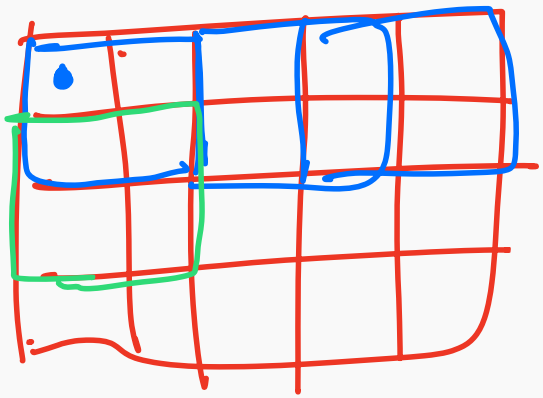
$$Y[:,:,:,m] = \sum_{c=1}^{C} W^{(m)}[:,:,:,c] * X[:,:,:,c]$$

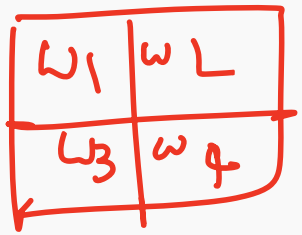
  - Using K filters $\Rightarrow Y \in \mathbb{R}^{D' \times H' \times W' \times K}$

Padding in CNNs: When and Why?

- **Padding** adds border pixels (usually zeros) so convolution can be applied near edges

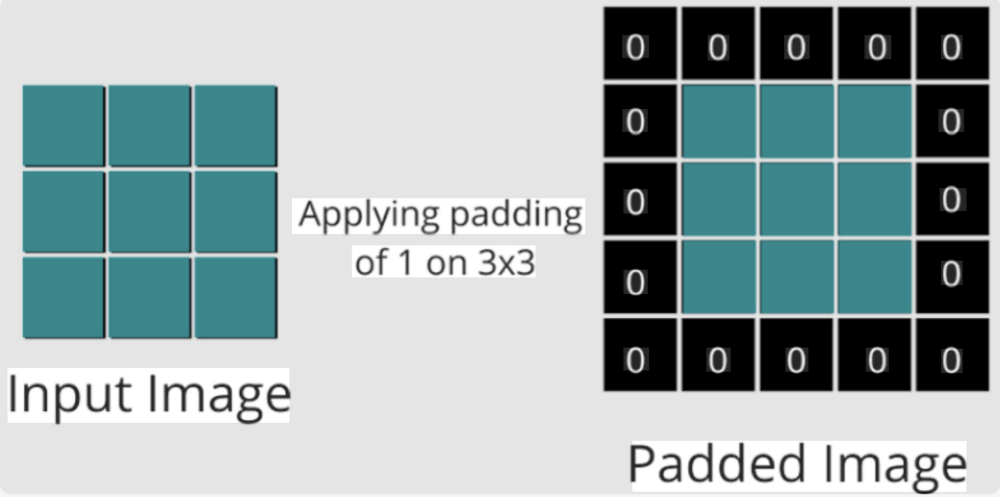






Padding in CNNs: When and Why?

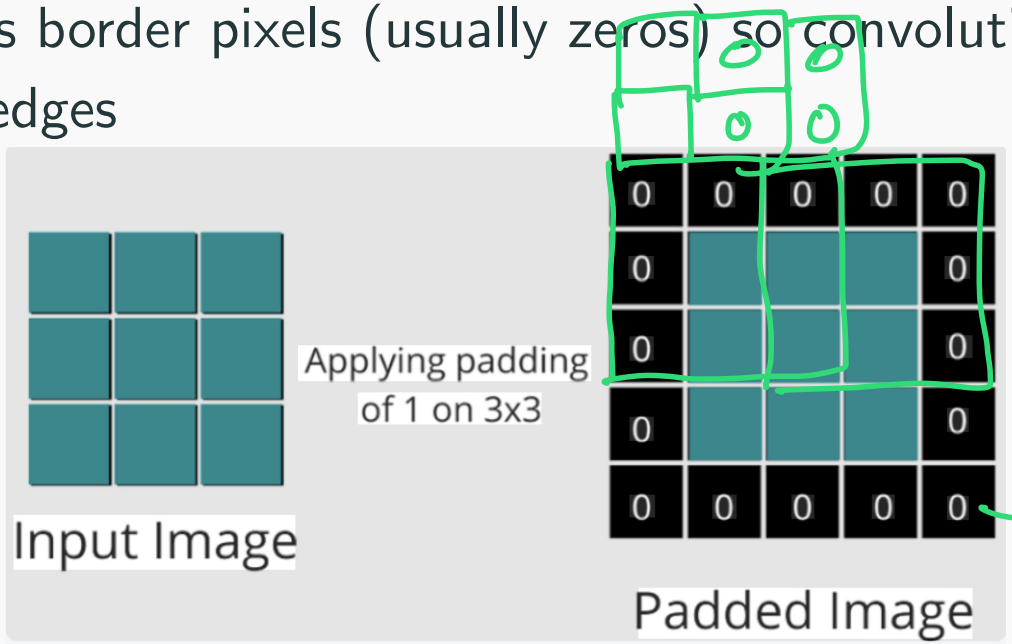
- **Padding** adds border pixels (usually zeros) so convolution can be applied near edges



- Why it matters:
  - avoids shrinking the feature map after every conv layer
  - preserves information at image borders (otherwise borders are seen fewer times)
  - helps keep shapes aligned across layers (useful in deep CNNs / skip connections)

Padding in CNNs: When and Why?

- **Padding** adds border pixels (usually zeros) so convolution can be applied near edges



3x3 ⇒ s=2 2x2

5x5 → 5x5

- Why it matters:
  - avoids shrinking the feature map after every conv layer
  - preserves information at image borders (otherwise borders are seen fewer times)
  - helps keep shapes aligned across layers (useful in deep CNNs / skip connections)
- Two types of padding:
  - **Valid** padding: $P = 0$ (no padding) $\Rightarrow$ output shrinks
  - **Same** padding: choose P to roughly preserve size

Activation Functions in CNNs: Why and When?

- **Why do we need them?**
  - Without activation functions, the network is just a **linear model**
  - Stacking linear layers $\Rightarrow$ still linear (no extra power)
  - Activations introduce **non-linearity** $\Rightarrow$ can learn complex patterns

Activation Functions in CNNs: Why and When?

- **Why do we need them?**
  - Without activation functions, the network is just a **linear model**
  - Stacking linear layers $\Rightarrow$ still linear (no extra power)
  - Activations introduce **non-linearity** $\Rightarrow$ can learn complex patterns
- **ReLU (Rectified Linear Unit)**

$$\mathrm{ReLU}(x) = \max(0, x)$$

  - Very simple and computationally efficient
  - Avoids vanishing gradients (unlike sigmoid/tanh)

Activation Functions in CNNs: Why and When?

- **Why do we need them?**
  - Without activation functions, the network is just a **linear model**
  - Stacking linear layers $\Rightarrow$ still linear (no extra power)
  - Activations introduce **non-linearity** $\Rightarrow$ can learn complex patterns
- **ReLU (Rectified Linear Unit)**

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} 0 & x < 0 \\ x & x > 0 \end{cases}$$

$\Rightarrow \sigma(w_1 x_1 + w_2 x_2 + \dots + w_9 x_9)$

  - Very simple and computationally efficient
  - Avoids vanishing gradients (unlike sigmoid/tanh)
- **Leaky ReLU**

$$\mathrm{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}, \qquad \alpha > 0 \quad \text{is a small constant}$$

$\alpha = 0.01$

  - Fixes "dead neuron" problem in ReLU
  - Allows small gradient when $x < 0$
- **When are they applied?**
  - Applied **after every convolution layer**: Typically `Conv` $\rightarrow$ `ReLU`

Batch Normalization: Why and How?

- **Why do we need BatchNorm?**
  - Deep networks are hard to train due to:
    - Changing distributions of activations (internal covariate shift)
    - Exploding / vanishing gradients
  - BatchNorm stabilizes and speeds up training

Batch Normalization: Why and How?

- **Why do we need BatchNorm?**
  - Deep networks are hard to train due to:
    - Changing distributions of activations (internal covariate shift)
    - Exploding / vanishing gradients
  - BatchNorm stabilizes and speeds up training
- **What does it do?**

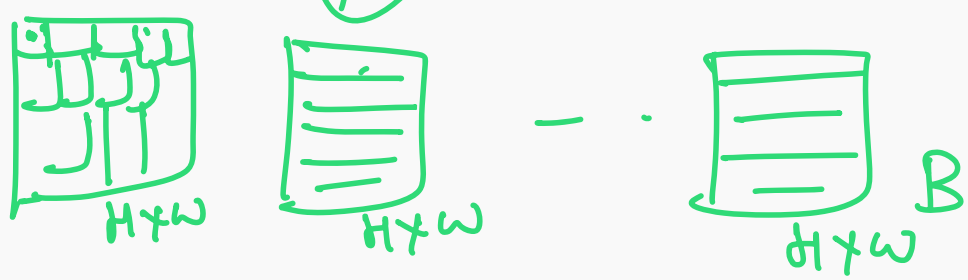
$$\Rightarrow \hat{x}_{ijb} = \frac{x_{ijb} - \mu_{ijb}}{\sqrt{\sigma_{ijb}^2 + \varepsilon}}, \quad i = 1 \ldots H, \quad j = 1 \ldots W$$

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

  - Normalizes activations using mini-batch statistics
  - Then rescales using learnable parameters (γ, β)

$$\mu_B = \frac{1}{b} \sum_{i=1}^{b} x_i$$

$$\sigma_B^2 = \frac{1}{b} \sum_{i=1}^{b} (x_i - \mu_B)^2$$



x_{ijb}

$$\Rightarrow \mu_{ijB} = \frac{1}{B} \sum_{b=1}^{B} x_{ijb}$$

$$\sigma_{ijB}^2 = \frac{1}{B} \sum_{b=1}^{B} (x_{ijb} - \mu_{ijB})^2 \rightarrow i = 1 \ldots H, \; j = 1 \ldots W$$

$B = 10$

Batch Normalization: Why and How?

- **Why do we need BatchNorm?**
  - Deep networks are hard to train due to:
    - Changing distributions of activations (internal covariate shift)
    - Exploding / vanishing gradients
  - BatchNorm stabilizes and speeds up training
- **What does it do?**

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

  - Normalizes activations using mini-batch statistics
  - Then rescales using learnable parameters (γ, β)
- **Why is it popular in CNNs?**
  - Faster convergence (fewer epochs)
  - Allows **higher learning rates**
  - Acts as a **regularizer** (less overfitting)
  - Makes training deep CNNs feasible

Batch Normalization: Why and How?

- **Why do we need BatchNorm?**
  - Deep networks are hard to train due to:
    - Changing distributions of activations (internal covariate shift)
    - Exploding / vanishing gradients
  - BatchNorm stabilizes and speeds up training
- **What does it do?**

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

  - Normalizes activations using mini-batch statistics
  - Then rescales using learnable parameters (γ, β)
- **Why is it popular in CNNs?**
  - Faster convergence (fewer epochs)
  - Allows **higher learning rates**
  - Acts as a **regularizer** (less overfitting)
  - Makes training deep CNNs feasible
- **Where is it applied?**
  - Typically: `Conv` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU`

Key Hyperparameters in Convolution

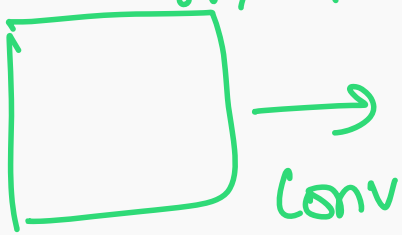
parameters

- **Kernel size** ($k_h \times k_w$): size of filter window (e.g., 3×3)
- **Stride** (S): step size when sliding the filter (e.g., $S = 2$ downsamples)
- **Padding** (P): zeros added at borders to control output size
  - **Valid**: no padding ($P = 0$)
  - **Same**: output size same as input
- **Number of filters** (K):
  - Determines number of output channels
- **(Optional) Dilation**:
  - Expands receptive field without increasing parameters

Key Hyperparameters in Convolution

- **Kernel size** ($k_h \times k_w$): size of filter window (e.g., 3×3)
- **Stride** (S): step size when sliding the filter (e.g., $S = 2$ downsamples)
- **Padding** (P): zeros added at borders to control output size
  - **Valid**: no padding ($P = 0$)
  - **Same**: output size same as input
- **Number of filters** (K):
  - Determines number of output channels
- **(Optional) Dilation**:
  - Expands receptive field without increasing parameters

input $H \times W \times C$



conv

$H' \times W' \times K$

Output Size Formula

$$H' = \left\lfloor \frac{H + 2P - k_h}{S} \right\rfloor + 1, \qquad W' = \left\lfloor \frac{W + 2P - k_w}{S} \right\rfloor + 1$$

Output: $X' \in \mathbb{R}^{H' \times W' \times K}$

- Example: $H = W = 32$, $k = 3$, $S = 1$, $P = 1 \Rightarrow H' = W' = 32$ (same)

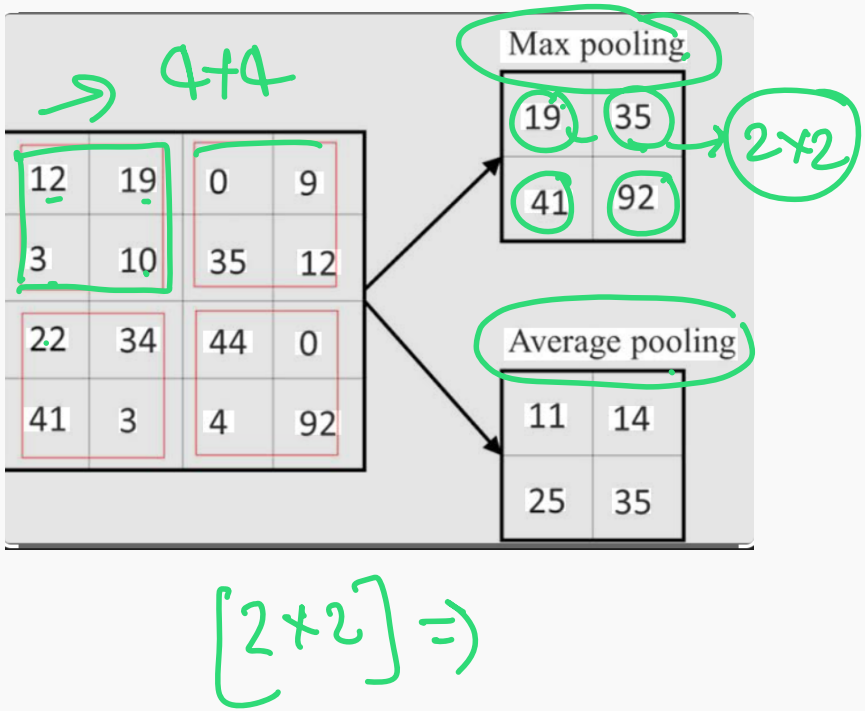
32×32 $\quad 3 \times 3$ $\quad P = 1$

Why Pooling?

- **Reduces spatial size**:
  - From $H \times W \to H' \times W'$ $\Rightarrow$ $H' << H$, $W' << W$
  - Reduces computation and memory
- **Provides translation invariance**:
  - Small shifts in input do not change output much
- **Increases receptive field**:
  - Each pooled value summarizes a larger region
- **Reduces overfitting**:
  - Acts as a form of downsampling / regularization

Pooling Types (and When to Use Them)

- **Max Pooling**
  - Takes the maximum value in each window
  - **Important when:** you care about *whether a feature exists* (edge/texture presence), and want robustness to small shifts
  - Common in early CNNs and in feature-extraction backbones
- **Average Pooling**
  - Takes the mean value in each window
  - **Important when:** you want *smoother* / less noisy activations (e.g., low-texture regions, denoising-like effects)



How Pooling Works

- Applied **independently on each channel**
- For input:

$$X \in \mathbb{R}^{H \times W \times C}$$

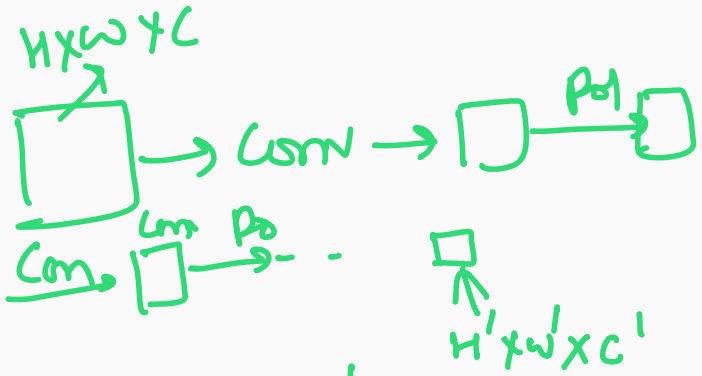
- A window of size $k \times k$ slides with stride s
- Output:

$$Y \in \mathbb{R}^{H' \times W' \times C}$$

- **Important:**
  - Number of channels C remains unchanged
  - Only spatial dimensions are reduced

Fully Connected (Dense) Layers i

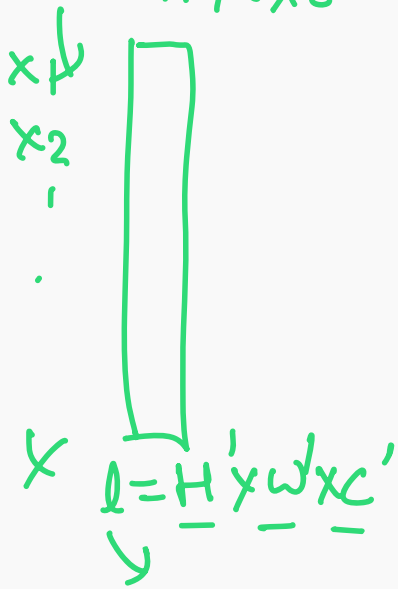
- A **Fully Connected (FC)** layer connects every input unit to every output unit
- **Role in CNNs:**
  - Placed near the **end of the network**
  - Combines all extracted features **globally**
  - Performs **final prediction / classification**



- **Input:**
  - Feature maps are **flattened** into a vector

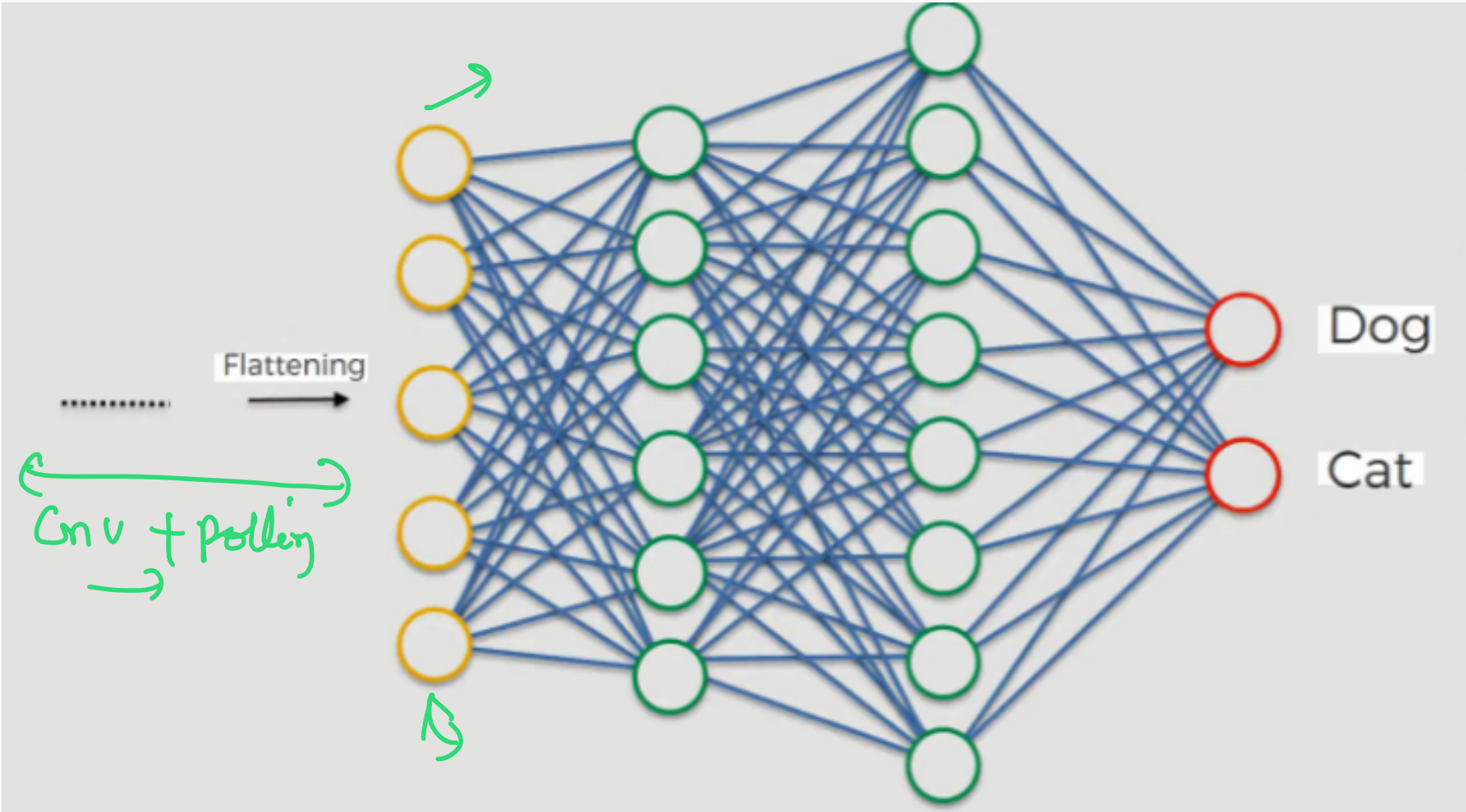
$$X \in \mathbb{R}^{H \times W \times C} \rightarrow \mathbf{x} \in \mathbb{R}^{n}$$

  where $n = HWC$



- **Interpretation:**
  - Learns **global combinations** of features
  - Acts like a standard **neural network layer**
  - Has **large number of parameters** (can cause overfitting)

Fully Connected Layers ii



Agenda

2. Convolutional Neural Networks (CNNs)

2.1 From MLP to CNN

2.2 Core Components of CNNs

Convolution

Pooling

Fully Connected Layers

2.3 Full CNN Pipeline

2.4 Training CNNs

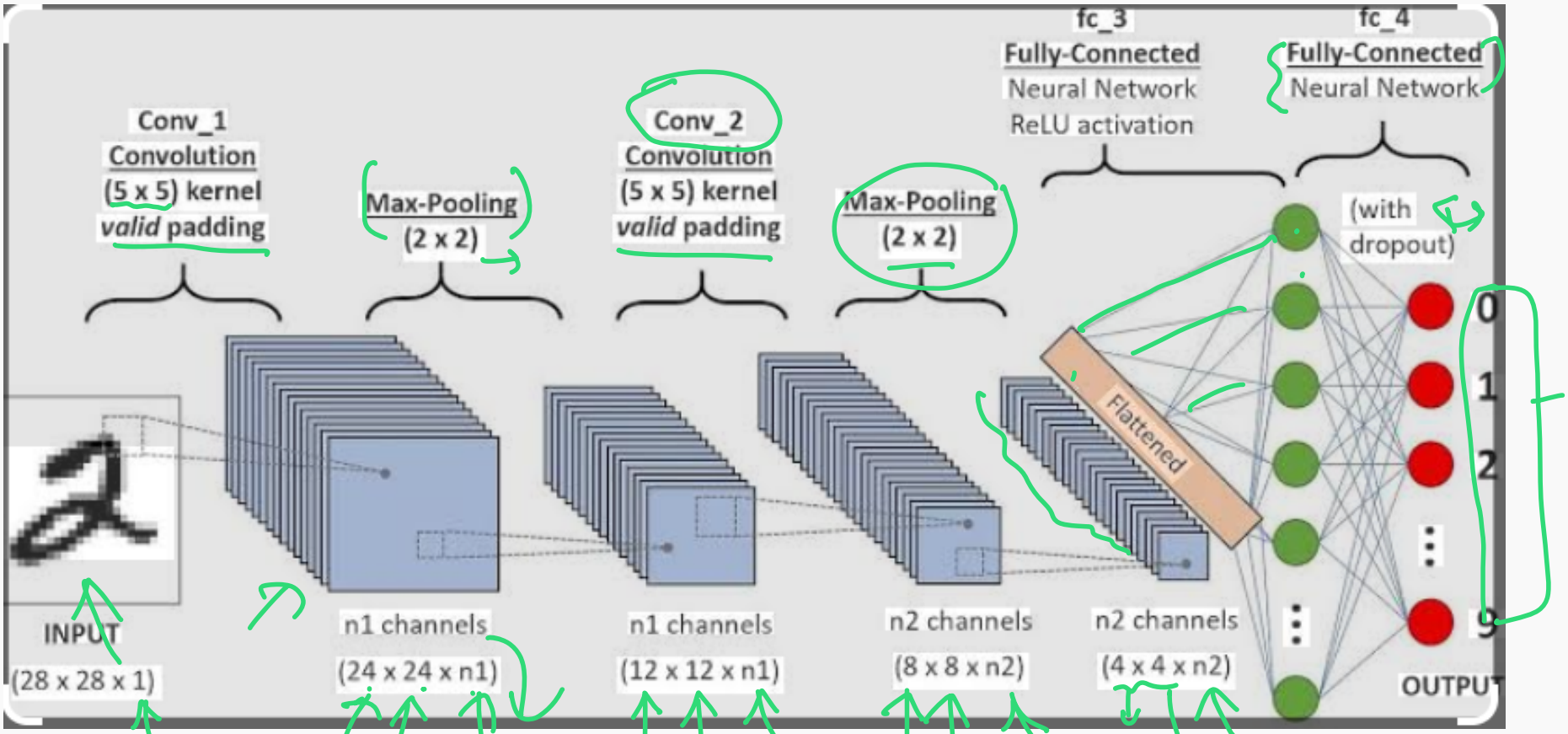
2.5 Practical Tips

2.6 Popular CNN Architectures

2.7 CNNs Lab

One Example of CNN Architecture

Input → Conv → ReLU → Pool
→ Conv → ReLU → Pool
→ Fully Connected → Softmax (for classification)



(28x28x1)

h', w

4x4xn2

→ 160

n2=10

n2=5